Chapter 1: Modern Javascript Essentials ..... 1
Let ..... 2
Const ..... 2
Objects ..... 3
This ..... 4
Arrow functions ..... 5
Destructuring ..... 6
Spread ..... 7
Classes ..... 8
Module ..... 11
Chapter 2: TypeScript for JavaScript Programmers ..... 12
Types by Inference ..... 13
Defining Types ..... 13
Composing Types – Union ..... 14
Composing Types – Generic ..... 15
Structural Type System ..... 16
Chapter 3: TypeScript - The Basics ..... 17
Static type checking ..... 18
Non-exception Failures ..... 18
Types for Tooling ..... 18
tsc, the TypeScript compiler ..... 19
Emitting with Errors ..... 19
Explicit Type ..... 20
Erased Types ..... 20
Downleveling ..... 21
Strictness ..... 22
noImplicitAny ..... 22
strictNullChecks ..... 23
Chapter 4: TypeScript - Everyday Types ..... 24

The premitives: string, number and boolean ..... 25
Array ..... 25
Any ..... 26
NoImplicitAny ..... 26
Type Annotations on Variables // Type annotations are used manualy specify the type of the variable ..... 27
Functions ..... 27
Return Type Annotations ..... 27
Object Types ..... 28
Optional Properties ..... 28
Anonymous Functions ..... 29
Union types and Defining Union Types ..... 29
Working with Union Types ..... 29
Type Aliases ..... 30
Interfaces ..... 31
Differences Between Type Aliases and Interfaces ..... 31
Type Assertions ..... 32
Literal Types ..... 33
Literal Inference ..... 34
null and undefined ..... 34
Non-null Assertion Operator (Postfix!) ..... 35
Enums ..... 35
Less Common Primitives ..... 36
Chapter 5: TypeScript – Narrowing ..... 36
typeof type guards ..... 37
Truthiness narrowing ..... 37
Equality narrowing ..... 38
The in operator narrowing ..... 39
instanceof narrowing ..... 40

Assignments ..... 41
Control flow analysis ..... 41
Using type predicates ..... 42
Discriminated unions ..... 43
The never type ..... 44
Exhaustiveness checking ..... 45
Chapter 6 : TypeScript - More on Functions ..... 46
Function Type Expressions ..... 47
Call Signatures ..... 47
Construct Signatures ..... 48
Generic Functions ..... 48
Inference ..... 49
Constraints ..... 49
Working with Constrained Values ..... 50
Specifying Type Arguments ..... 50
Guidelines for Writing Good Generic Functions ..... 51
Optional Parameters ..... 52
Optional Parameters in Callbacks ..... 52
Function Overloads ..... 53
Writing Good Overloads ..... 54
Declaring this in a Function ..... 55
Other Types to Know About ..... 55
void ..... 55
object ..... 55
unknown ..... 56
never ..... 56
Function ..... 56
Rest Parameters and Arguments ..... 57
Rest Arguments ..... 57

Parameter Destructuring ..... 58
Assignability of Functions ..... 58
Chapter 7: TypeScript - Object Types ..... 60
Optional Properties ..... 61
readonly Properties ..... 61
Index Signatures ..... 62
Extending Types ..... 63
Intersection Types ..... 64
Interfaces vs. Intersections ..... 65
Generic Object Types ..... 66
The Array Type ..... 67
The ReadonlyArray Type ..... 67
Tuple Types ..... 68
readonly Tuple Types ..... 69
Chapter 8: TypeScript - Type Manipulation ..... 69
Generics ..... 70
Generic function ..... 70
Generic type ..... 70
Generic Classes ..... 70
Generic constraint ..... 70
Type parametets in Generic constraints ..... 71
Using class type in generics ..... 71
Generic Default Types ..... 72
Varience Annotation ..... 72
Keyof Type Operator ..... 73
Typeof Type Operator ..... 74
Indexed Access Types ..... 74
Conditional Types ..... 74
Mapped Types ..... 75

Template Literal Types ..... 76
Chapter 9: TypeScript – Classes ..... 77
Class Members ..... 78
Fields ..... 78
Readonly ..... 79
Constructors ..... 79
Methods ..... 81
Getters / Setters ..... 81
Index Signatures ..... 82
Class Heritage ..... 82
Implements Clause ..... 83
Extend clause ..... 83
Overriding ..... 84
type-only field declaration ..... 85
Initialization order ..... 85
Extending Built-in types ..... 86
Member Visibility ..... 86
Access modifiers ..... 86
Cross-instance private access ..... 87
Caveats ..... 87
Static Members ..... 88
static Blocks in Classes ..... 90
Generic Classes ..... 90
this ..... 91
this at Runtime in Classes ..... 91
using arrow functions ..... 91
this parameter ..... 93
this type ..... 93
this-type based off-guard ..... 94

Parameter Properties ..... 95
Class Expressions ..... 96
abstract Classes and Members ..... 97
Relationships Between Classes ..... 97
Chapter 10: TypeScript – Modules ..... 98
Modules in TypeScript ..... 99
CommonJS Syntax ..... 100
TypeScript's Module Resolution Options ..... 101
TypeScript namespaces ..... 101
Import Module ..... 102
Export module - Named Export and Default Export ..... 102
Chapter 11: TypeScript – Interfaces ..... 103
Interfaces syntax ..... 104
Optional Property ..... 104
Readonly ..... 104
Function Type ..... 104
Index Signature ..... 105
Extending Interfaces ..... 105
Interface Merging ..... 105
Interfaces for Classes ..... 106
Extending Classes with Interfaces ..... 106
Interfaces with Generics ..... 107

# Chapter 1: Modern Javascript Essentials

## Let

```
// let used to store the data into variable

let message = "Hello!";

console.log(message);
//let has block scope and cannot be re-declared in same scope
if (true) {
  let test = true;
}
console.log(test);

//ReferenceError: test is not defined
```

## Const

```
/*
    const is used to declare and store the constant data (Unchangeble)
*/

// 1. Can't reassigned the variable after initial assignment
const PI = 3.14;
PI = 2.12;  //TypeError: Assignment to constant variable.



//  2. It is Block-scoped like let
if(true)
{
    const MAX_USER = 100;
}
console.log(MAX_USER); // ReferenceError: MAX_USER is not defined

// 3. Can't reassigned the array or object but it can be mutable
const person = { name: "Alice", age: 25 };
person.age = 26;  // Allowed
person.city = "London";  // Allowed

person = {}; // TypeError: Assignment to constant variable
```

## Objects

```
//declaration
let object = new Object();  //"object constructor" syntax
let object_2 = {};  // "object literal" syntax

let user = {
    name : "Mahesh",
    age : 23,
}

console.log("Initial user", user);

// Access the properties
console.log(user.name);
console.log(user.age);

//Modifies the existing properties
user.name = "Ronak";
console.log("modified user", user);

//Insert new properties
user.isAdmin = true;
console.log("User after property insertion" ,user);

//Delete property
delete user.isAdmin;
console.log("User after property deletion" ,user);

// how to use multiword property as key
let testObject = {
    name :  "Mahesh",
    age :  23,
    "like birds" : true,
}
console.log("Multiword property", testObject);

//Square bracket access
/* user.likes birds = true  // this would give a syntax error */
console.log(testObject["like birds"]);
```

```
testObject["like birds"] = false;
console.log(testObject["like birds"]);

delete testObject["like birds"];
console.log("testObject after key deletetion", testObject);

// in operator to check if propery exist in object
console.log( user.age === undefined ); // true means "no such property"
console.log( "age" in user ); // true, user.age exists
console.log( "blabla" in user ); // false, user.blabla doesn't exist

// For in loop in object
for (let key in user) {
  console.log( key );
  console.log( user[key] );
}
```

## This

```
// this refers to the object that is currently executing the function.

function showThis() {
    console.log(this);  // it give undefined
  }
  showThis();

  // 1. this Inside an Object Method
  // The function is called as a method of person.
  // So this points to the person object.
  const person = {
    name: "Alice",
    greet: function() {
      console.log(this.name);
    }
  };
  person.greet();


  // arrow function do not have their own this
  //they inherit from surrounding scope
```

```
  const personTest = {
    name: "Bob",
    greet: () => {
      console.log(this.name);
    }
  };
  personTest.greet(); // undefined
```

## Arrow functions

```
// Arrow function has shorter syntax than conventional

/*
----- conventional functiom
function sum(a, b)
{
    return a + b;
}
*/

// Arrow function
let sum = (a, b) => a+b;
console.log(sum(2,4));

//with single argument
let double = n => n * 2;
console.log(double(2))

let age = prompt("what is you age?", 18);

let welcome = (age > 18) ?
                () => console.log("you are above 18") :
                () => console.log("you are below 18");
welcome();

// arrow function as method in object
const person = {
    age : 19,
    name : "Mahesh",
    greeting : () => {
        console.log("Hello", this.name)
    }
```

```
};
person.greeting(); // --> Hello undefined : as arrow function does not have their
on this

//arrow function can access this of their surrounding(lexical) scope
const user = {
    age: 20,
    name: "Ronak",
    greeting(){
        //here this refers to the current object. i.e person
        console.log("outer function :", this.name);

        let inner = () => console.log("inner arrow function ", this.name);
        inner();
    }
}
user.greeting();
```

## Destructuring

```
// Destructuring assignment is special syntax to unpack array or object into
variables

// Array destructuring
const arr = ['Mahesh', 'Ronak'];
const storyArr = ["Julius", "Caesar", "Consul", "of the Roman Republic"];
let [firstName, lastName] = arr;
console.log(firstName);
console.log(lastName);

// Ignore elements using commas
let [name, , title] = storyArr;
console.log(title);  // --> Consul

//On iterables
// it is suger syntax to call for of on right values and assign to left value
using =
let [a, b, c] = "abc"; // ["a", "b", "c"]
let [one, two, three] = new Set([1, 2, 3]);

// ...(remaining elements)
let [name1, name2, ...rest] = storyArr;
```

```
console.log("length of rest", rest.length);
console.log("remaining element", rest);

// let [val1, val2] = [];
// val1 = undefined, va2 = undefined

//default values
let [val1 = "Mahesh", val2 = "Nanera"] = ["Meet"]
console.log("val1:", val1);
console.log("val2:", val2);



//Object destructuring
let options = {
  settingName: "Menu",
  width: 100,
  height: 200
};

// let {settingName, width, height} = options;
// console.log("options property values:", settingName, width, height);

let {settingName, ...remaining} = options;
console.log("width:", remaining.width);
console.log("height:" ,remaining.height);
```

## Spread

```
// Spread operator allows you to expand the elements of iterables (array, string
or object) into individual elements

// 1: Expanding an array
const numbers = [1,2,3];
console.log(...numbers);

// 2: Copying an array
const arr1 = [1,3,4];
const arr2 = [...arr1];
console.log("arr2:", arr2);
console.log(arr1 === arr2); //new array created only values are copied

// 3: Merging arrays
```

```
const a = ['a', 'b'];
const b = ['c', 'd'];
const c = [...a, ...b];
console.log("merged array c:", c);

const newNumbers = [5, ...numbers, 40];
console.log("new numbers",newNumbers);

// Spread with Objects
const person = { name: "Mahesh", age: 19 };
const personCopy = { ...person };
console.log("personCopy", personCopy);

// merged object
const contact = { email: "mahesh@mail.com" };
const merged = { ...person, ...contact };
console.log("merged object",merged);

const name = "Mahesh";
console.log([...name]);

// 4. Spread with Function Calls
function sum(a, b, c) {
  return a + b + c;
}
console.log("sum:" ,sum(...numbers));
```

## Classes

```
// Class declaration
class Person{
    // Constructor method – called when creating a new instance
    constructor(name, age){
        this.name = name;
        this.age = age;
    }

    //Instance method : Shared by all the instance
    greet(){
        console.log(`Hello my name is ${this.name} and I am ${this.age} years
old`);
    }
```

```
    //Static method: belongs to class itself
    static add(a, b) {
        return a + b;
    }
}

const person1 = new Person("Mahesh", 25);
person1.greet();
console.log(Person.add(2, 4));

//Inheritance
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound.`);
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name); // calls parent constructor
    this.breed = breed;
  }

  speak() {
    console.log(`${this.name} barks.`);
  }
}

const dog = new Dog("Tommy", "Bulldog");
dog.speak();

//Private fields
class BankAccount{
    #balance = 0;

    deposit(amount) {
        this.#balance += amount;
```

```
        console.log(`Deposited ₹${amount}`);
      }

      getBalance() {
        return this.#balance;
      }
}

const bankAccountObj = new BankAccount();
bankAccountObj.deposit(5000);
console.log("balance:", bankAccountObj.getBalance())
// console.log(bankAccountObj.#balance) --> gives error


//Getter and setter
class Student {
  constructor(name) {
    this._name = name;
  }

  // Getter
  get name() {
    return this._name.toUpperCase();
  }

  // Setter
  set name(value) {
    if (value.length < 3) {
      console.log("Name must have at least 3 letters!");
    } else {
      this._name = value;
    }
  }
}

// Usage
const student1 = new Student("Mahesh");

console.log(student1.name);

student1.name = "Al";
student1.name = "Rohit";
```

```
console.log(student1.name);
```

## Module

```
/*
      1. export keyword labels variable and function that should be accesible
outside the current module
      2. import allows the import of functionality from other module
*/

// Greeting.js
export function sayHi(name)
{
    console.log(`Hello and good morning from ${name}.`);
}

// User.js
// The import directive loads the module by path ./Greeting.js relative to the
current file
import {sayHi} from "./Greeting.js";
sayHi('Mahesh');

// Export and import class
// file: Vehicle.js
export class Car {
  start() {
    console.log("Car started!");
  }
}

export class Bike {
  start() {
    console.log("Bike started!");
  }
}

// Main.js
import {Car, Bike} from "./Vehicle.js";

const car = new Car();
car.start();
```

```
const bike = new Bike();
bike.start();
```

# Chapter 2: TypeScript for JavaScript Programmers

## Types by Inference

```
//Types by inferences means TypeScript automatically figures out

let message = "Hello there";
let age = 15;
let arr = ['Mahesh', 'Ronak'];

//message = 124; // Error Type 'number' is not assignable to type 'string'.
//arr.push(3)  // Argument of type 'number' is not assignable to parameter of
type 'string'.

// object type inference
let person = {
  name: "Mahesh",
  age: 25,
};

// person.name = 124; // Type 'number' is not assignable to type 'string'.

let mixedArray = ["Mahesh", 32];
mixedArray.push('Ronak');  //Allowed
mixedArray.push(12); //Allowed
mixedArray.push(true);  // Argument of type 'boolean' is not assignable to
parameter of type 'string | number'.
```

## Defining Types

```
interface Person{
    name: string,
    age: number
};

const person: Person = {
    // userName: "mahesh12", // Object literal may only specify known properties,
and 'userName' does not exist in type 'Person'.
    name: "Mahesh",
    age: 25,
    // age: "25"  // Type 'string' is not assignable to type 'number'.
}
```

```
console.log(person);

// Custom type creation
type PersonType = {
    name: string,
    age: number,
    email? : string // optional property
}

const person1: PersonType = {
    name: "Meet",
    age: 23,
    email: "meet@email.com"
}

console.log(person1);

class PersonProfile{
  name: string;
  age: number;

  constructor(name: string, age: number){
    this.name = name;
    this.age = age;
  }
}

const personProfileObj: Person = new PersonProfile("Shyam", 23);
console.log(personProfileObj);
```

## Composing Types – Union

```
//with union we can declare that type can be many of other types

type WindowStates = "open" | "closed" | "minimized";
type LockStates = "locked" | "unlocked";
type PositiveOddNumbersUnderTen = 1 | 3 | 5 | 7 | 9;

function describeWindowState(state: WindowStates) {
  switch (state) {
    case "open":
      return "The window is open.";
```

```
    case "closed":
      return "The window is closed.";
    case "minimized":
      return "The window is minimized.";
  }
}

console.log(describeWindowState("open"))
console.log(describeWindowState("closed"))
// console.log(describeWindowState("mahesh")) // Argument of type '"mahesh"' is
not assignable to parameter of type 'WindowStates'.

// Union in function parameters
function wrapInArray(obj: string | string[]) {
  if (typeof obj === "string") {
    return [obj];
  }
  return obj;
}

const names = ["Mahesh", "Ronak"];
const firstName = "Mahesh";

console.log(wrapInArray(names));
console.log(wrapInArray(firstName));
```

## Composing Types – Generic

```
type StringArray = Array<string>;
type NumberArray = Array<number>;
type ObjectWithNameArray = Array<{ name: string }>;

const studentNames: StringArray = ["Mahesh", "Ronak"];
// studentNames.push(100); //  Argument of type 'number' is not assignable to
parameter of type 'string'.
console.log(studentNames)

const people: ObjectWithNameArray = [
  { name: "Mahesh" },
  { name: "Ronak" },
  { name: "Shyam" },
```

```
//   { name: "Meet", age: 25}, // Object literal may only specify known
properties, and 'age' does not exist in type '{ name: string; }'.
];

console.log(people[0].name);

//Custom type with generic
type Box<T> = {
  value: T;
};

const numberBox: Box<number> = { value: 100 };
// const stringBox: Box<string> = { value: 100 }; // The expected type comes from
property 'value' which is declared here on type 'Box<string>'
const stringBox: Box<string> = { value: "Hello" };
const booleanBox: Box<boolean> = { value: true };

console.log(numberBox)

function identity<T>(value: T): T {
  return value;
}

console.log(identity<number>(10));
console.log(identity<string>("Hi"));
```

## Structural Type System

```
//In Structural type system - if two objects have the same structure than they
are considered to be of the same type.

interface Person {
    name: string;
    age: number;
  }

  interface Employee {
    name: string;
    age: number;
  }

  let person: Person = { name: "Mahesh", age: 25 };
```

```
  let employee: Employee = person;

  //If works even Person and Employee are different interfaces. Because they both
have same shape

  // two objects have the same set of properties with the same types, they are
considered compatible.

  interface Point2D {
    x: number;
    y: number;
  }

  interface Point3D {
    x: number;
    y: number;
    z: number;
  }

  let point2D: Point2D = { x: 10, y: 20 };
  let point3D: Point3D = { x: 10, y: 20, z: 30 };

  point2D = point3D;  // allowed as Point3D has everything that point2D neeed
  // point3D = point2D;  //Property 'z' is missing in type 'Point2D' but required
in type 'Point3D'.

  // Type checking for functions
  type Log = (message: string) => void;
  type Warn = (msg: string) => void;

  let log: Log = (m) => console.log(m);
  let warn: Warn = log;

  log("Log");
  warn("Warn you");
```

# Chapter 3: TypeScript - The Basics

## Static type checking

```
// TypeScript performs static type-checking --> It analyze the code before
executing and find type-related errors

function sum(a: number, b:number){
    return a + b;
}

console.log(sum(2, 5));
console.log(sum('3', 5));  //Argument of type 'string' is not assignable to
parameter of type 'number'.
```

## Non-exception Failures

```

//In TypeScript Non-exception failure refers to the error that TypeScript detects
at compile-time

function greet(name: string)
{
    return "Hello " + name.toUpperCase();
}

// Argument of type 'number' is not assignable to parameter of type 'string'.
//While in js it says that name.toUpperCase  is not a function --> runtime
exception
console.log(greet(24))


let user = { name: "Mahesh", age: 25 };
user.email = 'mahesh@temp.com';  // Property 'email' does not exist on type
'{ name: string; age: number; }'.  while in js it return undefined

const announcement = "Hello World!";
announcement.toLocalLowerCase();  // Property 'toLocalLowerCase' does not exist
on type '"Hello World!"'. Did you mean 'toLocaleLowerCase'?
```

## Types for Tooling

```
//TypeScript start suggesting the properties that you might want to use
import express from "express";
const app = express();

app.get("/", function (req, res) {
  res.send   //Here you can get the suggestion like send, sendDate,sendFile etc.
//       ^|
});

app.listen(3000);
```

## tsc, the TypeScript compiler

```
//tsc -> TypeScript compiler that do following things
// 1. Checks our TypeScript code for type error(static type checking)
// 2. Converts the .ts file into .js file

function greet(name: string)
{
    return "Hello " + name.toUpperCase();
}

console.log(greet("Mahesh"));  // Hello MAHESH
console.log(greet(45));   // Argument of type 'number' is not assignable to
parameter of type 'string'.
```

## Emitting with Errors

```
// TypeScript by default generates the .js file even if it contains type errors
// that can cause runtine crashes

tsc app.js   // --> type check + generates the .js file even if type error exist

//Preventions
//1. tsconfig.json
{
    "compilerOptions": {
        "noEmitOnError": true
    }
}
```

```
//2. commnd line
tsc app.js --noEmitOnError   // --> while not genarate the .js file if type error exist
```

## Explicit Type

```
// In TypeScript we can manualy specify the type of variable, function or return value
//eg.

let userName: string = "Mahesh";
let age: number = 23;
let isLoggedIn: boolean = true;

// function parameteres
function greet(name: string, date: Date)
{
    return "Hello " + name.toUpperCase() + ", today is " + date.toLocaleString();
}



//return type
function sum(a: number, b:number) : number{
    return a+b;
}

// array
let nums: number[] = [1, 2, 3];
let names: string[] = ['Mahesh', 'Ronak'];

//object
let user: { name: string; age: number } = {
  name: "Mahesh",
  age: 25,
};

//union type
let id: string | number;
id = "A123";
id = 101;
id = true;   // Type 'boolean' is not assignable to type 'string | number'.
```

## Erased Types

```
// After compilation of typescript file the Type Annotations are erased in the
emitted .js file

//eg.
//greet.ts
function greet(name: string): string{
    return "Hello " + name;
}

let age: number = 25;

//after compilation
//greet.js
function greet(name)
{
    return "Hello " + name;
}

let age = 25;

// same with interfaces
//user.ts
interface User {
  name: string;
  age: number;
}

function printUser(user: User) {
  console.log(user.name + " is " + user.age + " years old");
}

//user.js
function printUser(user) {
  console.log(user.name + " is " + user.age + " years old");
}
```

## Downleveling

```
// tsc compiles the typescript file into older version of javascript like
ECMAScript 3 or ECMAScript 5 ---> It is called downleveling
```

```
//eg. greet.ts
let name: string = "Mahesh";
let date: Date = new Date();
console.log(`Hello ${name}, today is ${date.toDateString()}!`);

//greet.js
let name = "Mahesh";
let date = new Date();
console.log("Hello ".concat(name, ", today is ").concat(date.toDateString(),
"!"));



//To Prevent --> use --target keyword
// tsc --target es2015 greet.ts
```

## Strictness

```
// strictness means how tightly compiler checks for type correctness in code
// strict mode : ON --> catch every possible bugs at compile time
// strict mode : OFF --> allow risky and loosly type code

//setting
//tsconfig.json
// {
//     "compilerOptions": {
//         "strict": true
//     }
// }
```

## noImplicitAny

```
// noImplicitAny flag of compiler prevent variables, functions, return values
from automatically getting the type any if type is not specified.

// tsconfing.json
// {
//     "compilerOptions":{
//         "noImplicitAny" : true,  // or "strict" : true
//     }
// }
```

```
//Without noImplicitAny
function greet(name){
    return "Hello " + name;
}

console.log(greet("Mahesh"));  // Hello Mahesh
console.log(greet(45));     // Hello 45
console.log(greet(true));  // Hello true

//With noImplictAny
// Parameter 'name' implicitly has an 'any' type.

//Fix
function greet(name: string) {
  console.log("Hello " + name);
}



//Variables without noImplicitAny
let data;
data = 10;
data = "Mahesh";
data = true;

// with noImplictAny
//Error: Variable 'data' implicitly has an 'any' type.

// fix
let data: number;
```

## strictNullChecks

```
// strictNullChecks forces you to explicitly handle null and undefined before
using variables.

//tsconfig.json
// {
//   "compilerOptions": {
//     "strictNullChecks": true  //Or "strict" : true
//   }
// }
```

```
// Without strictNullChecks
// let name: string = null;  // Allowed
// console.log(name.toUpperCase()); //Runtime error: Cannot read property
'toUpperCase' of null

// With strictNullChecks: true
let name: string = null; // Error: Type 'null' is not assignable to type 'string'

// function greet(name: string) {
//   console.log("Hello " + name.toUpperCase());
// }

// greet(null);
// Error: Argument of type 'null' is not assignable to parameter of type 'string'

//Fix
function greet(name: string | null) {
  if (name) {
    console.log("Hello " + name.toUpperCase());
  } else {
    console.log("Hello stranger");
  }
}

// Optional Parameters
// function printLength(text?: string) {
//   console.log(text.length); // Error: Object is possibly 'undefined'
// }

// Fix
function printLength(text?: string) {
  if (text !== undefined) {
    console.log(text.length);
  }
}
```

# Chapter 4: TypeScript - Everyday Types

## The premitives: string, number and boolean

```
// Three very commonly used primitive types in js
// primitive means it store single value
// string, number(int of float), boolean

//equivalent typescript types are
let personName: string = "Mahesh";
let age: number = 23;
let amount: number = 3499.99;
let isLoggedIn: boolean = true;
```

## Array

```
// we can declare the array using
// 1. type[]
//2. Array<type>

interface User{
    name: string,
    age: number
};

let nums1: number[] = [1, 2, 3];
let names1: string[] = ['Mahesh','Meet'];
let boolArray1: boolean[] = [true, false];
let objArray1: {name: string, age: number}[] = [
        {name: 'Mahesh', age: 25},
        {name: 'Meet', age: 23},
    ];



// Array<Type>
let nums2: Array<number> = [1, 2, 3];
let names2: Array<string> = ['Mahesh','Meet'];
let boolArray2: Array<boolean> = [true, false];
let objArray2: Array<User> = [
        {name: 'Mahesh', age: 25},
        {name: 'Meet', age: 23},
    ];
```

## Any

```
// When variable if typed as any then, TypeScript turns off all type checking for
that variable



let value: any = "Hello";
value = 42;        // Allowed
value = true;      // Allowed
value = { a: 1 };  // Allowed



//Any disables the Type checking for that variable
let data: any = "Mahesh";

console.log(data.toUpperCase()); // Works (string)
data = 123;
console.log(data.toUpperCase()); // Runtime error: data.toUpperCase is not a
function
```

## NoImplicitAny

```
// noImplicitAny flag of compiler prevent variables, functions, return values
from automatically getting the type any if type is not specified.

// tsconfing.json
// {
//     "compilerOptions":{
//         "noImplicitAny" : true,  // or "strict" : true
//     }
// }

//Without noImplicitAny
function greet(name){
    return "Hello " + name;
}

console.log(greet("Mahesh"));  // Hello Mahesh
console.log(greet(45));     // Hello 45
console.log(greet(true));  // Hello true
```

```
//With noImplictAny
// Parameter 'name' implicitly has an 'any' type.
```

## Type Annotations on Variables

```
// Type annotations are used manualy specify the type of the variable

let message: string = "Mahesh";
let age: number = 23;
let isLoggedIn: boolean = true;
let numArr: number[] = [1, 3, 4];

const user: {name: string, age: number} = {
    name: "Mahesh",
    age: 23
}

interface User{
    name: string,
    age: number
}

const person: User = {
    name: "Meet",
    age: 34
}
```

## Functions

```
// Type declaration on Function

// Parameter Type Annotations
function add(a: number, b: number){
    return a + b;
}
```

## Return Type Annotations

```
// Return Type Annotations
```

```
function greet(name: string, date: Date) : string{
    return `Hello ${name}, today is ${date.toDateString()}`
};
```

## Object Types

```
// Object type declaration

interface User{
    name: string,
    age: number,
}

let user: {name: string, age: number} = {
    name: "Mahesh",
    age: 25
}

function greet(user: User){
    return `Hello I am ${user.name} and I am ${user.age} years old.`
}

console.log(greet({name: "Ronak", age: 23}));
```

## Optional Properties

```
// Optional properties

interface Person{
    firstName: string,
    lastName?: string  // Optional
}

function printName(person: Person){
    // if (obj.last !== undefined) {
    //     console.log(person.lastName.toUpperCase());
    // }

    console.log(person.lastName?.toUpperCase());
}
```

```
let person1: Person = {firstName: "Mahesh", lastName: "Nanera"};
let person2: Person = {firstName: "Ronak"};

printName(person1);
printName(person2);  //returns undefined as person2.lastName is undefined
```

## Anonymous Functions

```
// contextual typing – the type is inferred from where the function is used.

const names = ["Mahesh", "Ronak", "Raj"];

names.forEach(function(name) {  // type of name is string as names is the array
of string
  console.log(name.toUpperCase());
});

names.forEach((s) => {    // type of s is string as names is the array of string
  console.log(s.toUpperCase());
});
```

## Union types and Defining Union Types

```
// Using Union type typescript allow us to define new types from existing types

function print(value: number | string){
    console.log("Values is:", value)
}

print("Mahesh");
print(25)
// print(true)  // Argument of type 'boolean' is not assignable to parameter of
type 'customType'.
```

## Working with Union Types

```
// Union type allow the operation if that are valid for all the members

// function print(value: number | string){
//     console.log("Values is:", value.toUpperCase());
```

```
//     //Property 'toUpperCase' does not exist on type 'string | number'.
Property 'toUpperCase' does not exist on type 'number'.
// }

function print(value: number | string){
    if(typeof value === "string"){
        console.log("Value is", value.toUpperCase());
    }
    else
    {
        console.log("Value is", value);
    }
 }

 print("Mahesh");
 print(52);

 // if all the types in union has common property then that can be used directly

 function getFirstThree(arr: string[] | string){
     return arr.slice(0, 3);
 }

 console.log(getFirstThree("Mahesh"));
 console.log(getFirstThree(['Mahesh', 'Ronak', 'Shyam', 'Meet']))
```

## Type Aliases

```
// Type Alias is the name for the any type
// we can create the customTypes using type Alias and use it

type stringOrNumber = string | number;
type User = {name: string, age: number, email?: string};

function greet(user: User){
    console.log(`Hello my name is ${user.name} and I am ${user.age} years old.`);
};

let user1: User = {name: "Mahesh", age: 23, email: "mahesh@temp.com"};
let user2: User = {name: "Ronak", age: 22};
```

```
greet(user1);
greet(user2);
```

## Interfaces

```
// Type Alias is the name for the any object type
// we can create the customTypes using type Alias and use it

interface User{
    name: string,
    age: number,
    email?: string
}

function greet(user: User){
    console.log(`Hello my name is ${user.name} and I am ${user.age} years old.`);
};

let user1: User = {name: "Mahesh", age: 23, email: "mahesh@temp.com"};
let user2: User = {name: "Ronak", age: 22};

greet(user1);
greet(user2);
```

## Differences Between Type Aliases and Interfaces

```
//Difference between Interfaces and Type alias

// 1. Type alias can define any type - primitive, union,object while Interfacs
describes object shapes and class contracts
// 2. Interfaces can be extended but type can't be merged

//eg.
interface Animal{
    name: string
}

interface Dog extends Animal{
    canBark: boolean;
```

```
}

const pitbull: Dog = {name: "Tommy", canBark: true};
console.log(pitbull);

// Extending type with intersaction
type Bike = {
  name: string;
}

type Sp125 = Bike & {
    hasDiscBreak: boolean;
}

const sp125: Sp125 = {name: "bhalu", hasDiscBreak: true};
console.log(sp125);

//Merging Interface
interface Animal{
    isWild?: boolean
}

const rabbit: Animal = {name: "Rabbit", isWild: false};
console.log(rabbit);


//Merging type
// Duplicate identifier 'Bike'.
// type Bike = {
//     engineCapacity: number
// }
```

## Type Assertions

```
// Type assertion is the way to tell the compiler that "Trust me, I know the type
of this variable"

let value: unknown = "Mahesh";
// let len1 = value.length;  //Property 'length' does not exist on type
'unknown'.
let len = (value as string).length; // for
console.log(len);
```

```
let num = 10;
// let userName = num as string; //Conversion of type 'number' to type 'string'
may be a mistake because neither type sufficiently overlaps with the other. If
this was intentional, convert the expression to 'unknown' first.

let userName = num as unknown as string;
```

## Literal Types

```
// In TypeScript Literal values represent the exact values as type



//String literals
let direction: "left" | "right" | "up" | "down";

direction = "left";
direction = "right";
// direction = "forward";  //Type '"forward"' is not assignable to type '"left" |
"right" | "up" | "down"'.

//Number Literals
function compare(a: string, b: string): -1 | 0 | 1 {
  return a === b ? 0 : a > b ? 1 : -1;
}

//Bolean literals
type YesOrNo = true | false;

let answer: YesOrNo;
answer = true;
answer = false;
// answer = "true";  //Type 'string' is not assignable to type 'YesOrNo'.

//Literal types with objct
type DoorState = "open" | "closed";
type LockState = "locked" | "unlocked";

interface Door {
  state: DoorState;
```

```
  lock: LockState;
}

const frontDoor: Door = {
  state: "open",
  lock: "locked"
};
```

## Literal Inference

```
let role = "admin";   // Inferes the type as string
role = "user";  // allow

const permission = "read";  //inferes type as "read"
// permission = "write"; //Cannot assign to 'permission' because it is a
constant.

// Object to treat as const
const userObj = {name: "Mahesh", age: 25};
userObj.age = 22;  // allowed

const userObj2 = {name: "Mahesh", age: 25} as const;
userObj2.age  = 44;   // annot assign to 'age' because it is a read-only
property.
```

## null and undefined

```
//TypeScript has 2 primitive type that indicate the Absence of value or
uninitialized
//undefined and null

//strictNullChecks : ON  --> we cannot assign null --> then we have to add the
null using union type and handle the condition for that null

// let name: string = null;   // throws error
let userName: string | null | undefined;
userName = "Mahesh";
userName = null;
userName = undefined;
```

```
function print(userName : string | null | undefined){
    if(userName)
    {
        console.log(userName);
    }
    else if(userName === null)
    {
        console.log("userName has null value");
    }
    else{
        console.log("userName has undefined value")
    }
}
```

## Non-null Assertion Operator (Postfix!)

```
// On enable the strictNullChecks TypeScript forces us to handle values that
might be null or undefined

let userName: string | null = "Mahesh";
// console.log(name.length); // Error: Object is possibly 'null'

//Not null assertion means telling compiler that I know that values is not null
or undefined
// It just stope the type checking for the null value
//userName! --> it tell compiler that it has type string.
console.log(userName!.length);
```

## Enums

```
// Enum is TypeScript feature that lets you define a set of named constants
// their values are numeric starting from 0.

enum Direction {
    Up,
    Down,
    Left,
    Right
  }

  let direction: Direction = Direction.Up;
  console.log(direction);
```

```
console.log(Direction.Down)

//Reverse lookup
console.log(Direction[0]) // Up

// Custom Values in Numeric Enums
enum Status {
  Pending = 1,
  InProgress = 3,
  Completed = 5
}

// String Enums
enum DirectionMap {
  Up = "UP",
  Down = "DOWN",
  Left = "LEFT",
  Right = "RIGHT"
}

console.log(DirectionMap.Down); // "DOWN"
```

## Less Common Primitives

```
//big int

let big: bigint = 9007199254740991n;  // 'n' suffix makes it a bigint literal
let bigger: bigint = BigInt(100);
let sum: bigint = big + bigger;

console.log(sum)

//Symbol -- it used to create the unique identifier
const sym1 = Symbol("id");
const sym2 = Symbol("id");

console.log(sym1 === sym2);
```

# Chapter 5: TypeScript – Narrowing

## typeof type guards

```
// Narrowing --> Refine a variable's type to a more specific type based on
runtime check

//Type Guard --> it is a way to tell typescript that "inside this block, the
variable has specific type"

//typescript uses typeof operator to narrow down the union type like string |
number

//eg.

//Hre value has type string | number | boolean
function print(value: string | number | boolean){
    if(typeof value === "string"){
        // here value has specific type string
        console.log("Length of value is:", value.length);
    }
    else if(typeof value === "number"){
        // here value has specific type number
        console.log("The value is:", value.toFixed(2));
    }
    else{
        // here value has specific type boolean
        console.log(value ? "TRUE" : "FALSE");
    }
}

print("Mahesh");
print(45.09098998);
print(false);
```

## Truthiness narrowing

```
//Truthiness --> In js every value is either truthy or falsy
// falsy values are --> false, 0, "", null, undefined, Nan
// all other values are truthy (like "Mahesh", 123, [], {} etc)
```

```
//Typescript uses truthiness of value to narrow down the union type that contains
null or undefined

function printMessage(msg?: string) {
    if (msg) {
      //msg is truthy → narrowed to string
      console.log("Message:", msg.toUpperCase());
    } else {
      // msg is undefined or empty string (falsy)
      console.log("No message provided");
    }
  }

  printMessage("Hello"); // Message: HELLO
  printMessage();        // No message provided


  function greet(name: string | null | undefined) {
    if (!name) {
      console.log("Hello, stranger!");
    } else {
      console.log("Hello,", name.toUpperCase());
    }
  }

  greet("Mahesh");  // Hello, MAHESH
  greet(undefined); // Hello, stranger!
  greet(null);      // Hello, stranger!
```

## Equality narrowing

```
// Equity narrowing --> narrow down the type using equality or inequality
comparison — like ===, !==, ==, or !=.

function compare(a: string | number, b: string | boolean) {
    if (a === b) {
      // a and b must both be string (only common type)
      console.log(a.toUpperCase());
    } else {
      console.log("They are not equal");
    }
  }
```

```
compare(1, true);
compare("Mahesh", "Nanera");
compare("Mahesh", "Mahesh");

// Checking Against null or undefined
function greet(name?: string | null) {
  if (name == null) {
    // true for both null and undefined
    console.log("Hello, Stranger!");
  } else {
    // here name is narrowed to string
    console.log("Hello,", name.toUpperCase());
  }
}

greet("Mahesh");
greet(null);
greet(undefined);
```

## The in operator narrowing

```
// in operator checks whether a property exists in an object
// When use in operator typescript narrow down to only those types in union for
which the property is being checked by in operator
// if property is checked for string then it narrow down to string

type Car = {brand: string, speed: number};
type Bike = {brand: string, gear: number};

function getVehicleInfo(vehical: Car | Bike){
    if("speed" in vehical){
        // Vehical is narrow down to Car
        console.log("Car speed:", vehical.speed);
    }
    else{
        //Vehical is narrow down to Bike
        console.log("Bike gear:", vehical.gear);
    }
}
```

```
type Developer = { name: string; programmingLanguage: string };
type Designer = { name: string; tool: string };
type Employee = Developer | Designer | null;

function describeEmployee(emp: Employee) {
  if (emp && "programmingLanguage" in emp) {
    console.log(emp.name, "is a developer");
  } else if (emp && "tool" in emp) {
    console.log(emp.name, "is a designer");
  } else {
    console.log("No employee info");
  }
}
```

## instanceof narrowing

```
// instanceof checks whether a object is an instance of a class.
// typescript uses instanceof type guard to narrow the type

class Dog{
    bark(){
        console.log("Dog is barking!");
    }
}

class Cat{
    meow(){
        console.log("Cat is meowing!");
    }
}

function makeSound(animal: Dog | Cat){
    if(animal instanceof Dog){
        animal.bark();
    }
    else{
        animal.meow();
    }
}

const dog = new Dog();
```

```
const cat = new Cat();
makeSound(dog);
makeSound(cat);

// With build in Classes

function printValue(value: Date | string) {
  if (value instanceof Date) {
    // narrowed to Date
    console.log("Year:", value.getFullYear());
  } else {
    // narrowed to string
    console.log("String length:", value.length);
  }
}

printValue(new Date());
printValue("Mahesh");
```

## Assignments

```
// On assignment to variable typescript narrows the type based on the right side
of assignment

let x = Math.random() < 0.5 ? 10 : "Hello";

x = 1;
console.log(x);

x = "Mahesh";
console.log(x);

x = true;  //Type 'boolean' is not assignable to type 'string | number'.
```

## Control flow analysis

```
// Control flow analysis --> TypeScript continuously tracks variable's type as
code run through different logical paths - (i.e. if, else, switch etc.)

function example() {
    let x: string | number | boolean;
```

```
    x = Math.random() < 0.5;
    console.log(x);   // Type boolean

    if (Math.random() < 0.5) {
      x = "hello";
      console.log(x); // Type string
    } else {
      x = 100;
      console.log(x);   // Type number
    }

    return x;     // Type is string | number | boolean;
  }
```

## Using type predicates

```
// Type Predicates --> are the special return type used in function
// --> It tells typescript that if this function return true then parameter is of
specific type
// It is used to create the custom type guard to narrow the types

//eg.
function isString(value: unknown) : value is string{
    return typeof value === "string";
}

function printValue(value: unknown){
    if(isString(value)){
        //treated as string
        console.log(value.toUpperCase());
    }
    else{
        console.log(value);
    }
}

printValue("Mahesh");
printValue(123);

//Type predicate with object
function isPerson(obj: any): obj is { name: string } {
```

```
  return typeof obj === "object" && obj !== null && "name" in obj;
}

function greet(value: unknown) {
  if (isPerson(value)) {
    console.log("Hello, " + value.name);
  } else {
    console.log("Not a person");
  }
}

// Type predicate with interfaces
interface Dog {
  kind: "dog";
  bark(): void;
}

interface Cat {
  kind: "cat";
  meow(): void;
}


function isDog(animal: Dog | Cat): animal is Dog {
  return animal.kind === "dog";
}

function makeSound(animal: Dog | Cat) {
  if (isDog(animal)) {
    animal.bark();
  } else {
    animal.meow();
  }
}
```

## Discriminated unions

```
// Discriminated union is the union of the object types where each type shares a
common property with literal type unique to that
// TypeScript uses that common property to narrow types

interface Circle{
```

```
    kind: "circle",      //Discriminant
    radius: number,
}

interface Square{
    kind: "square",     //Discriminant
    sideLength: number,
}

type Shape = Circle | Square;    //Discriminated Union

function getArea(shape: Shape) : number{
    if(shape.kind === "circle"){
        return Math.PI * shape.radius**2;
    }
    else{
        return shape.sideLength**2;
    }
};

const circle: Circle = {kind: "circle", radius: 30};
const square: Square = {kind: "square", sideLength: 15};

console.log("Circle area:", getArea(circle));
console.log("Square area:", getArea(square));
```

## The never type

```
// The never type represents values that never occur.
//It is the way to tell TypeScript that "This code path can’t ever happen".

interface Circle{
    kind: "circle"
    radius: number,
}

interface Square{
    kind: "square",
    sideLength: number,
}

type Shape = Circle | Square;
```

```
function getArea(shape: Shape) : number{
    switch(shape.kind){
        case "circle":
            return Math.PI * shape.radius**2;
        case "square":
            return shape.sideLength**2;
        default:
        // This is piece of code is not reachable as TS knows that shape can't be
anything else except the case that we have handle
          const _exhaustiveCheck: never = shape;
          return _exhaustiveCheck;
    }
};

const circle: Circle = {kind: "circle", radius: 30};
const square: Square = {kind: "square", sideLength: 15};

console.log("Circle area:", getArea(circle));
console.log("Square area:", getArea(square));
```

## Exhaustiveness checking

```
// The never type represents values that never occur.
// Exhaustive checking means making sure all possible cases of a union type are
handled in your code.
// If you forget a case, it might cause runtime errors or unexpected behavior.
// Exhaustive checking ensures type safety and code completeness.
// No types are assignable to never

interface Circle{
    kind: "circle"
    radius: number,
}

interface Square{
    kind: "square",
    sideLength: number,
}

interface Triangle{
    kind: "triangle",
```

```
    length: number
}

type Shape = Circle | Square | Triangle;

function getArea(shape: Shape) : number{
    switch(shape.kind){
        case "circle":
            return Math.PI * shape.radius**2;
        case "square":
            return shape.sideLength**2;
        default:
          const _exhaustiveCheck: never = shape;    // Type 'Triangle' is not
assignable to type 'never'.
          return _exhaustiveCheck;
    }
};

const traingle: Triangle = {kind: "triangle", length: 15};

console.log("Triangle area:", getArea(traingle));
```

# Chapter 6 : TypeScript - More on Functions

## Function Type Expressions

```
// Function Type expressions --> Shape of a function. What it takes as parameters
and what it returns

function greet(name: string): string{
    return `Hello my name is ${name}`;
}

// using type alias
type GreetFn = (name: string, date: Date) => string;  // Function Type Expression

const greeting: GreetFn = (name, date) => {
    return `My nname is ${name} and today is ${date.toDateString()}`;
}

console.log(greeting("Mahesh", new Date()));

// Function Type as Return Value
type Multiply = (a: number, b: number) => number;

function createMultiplier(multiplier: number): Multiply {
  return (a, b) => (a * b) * multiplier;
}

const doubleMultiply = createMultiplier(2);
console.log(doubleMultiply(2, 3));
```

## Call Signatures

```
// Call signature describe how function can be called.
//It defines parameters and return type of object
//mainly used for object type or interface type

interface Add{
    (a: number, b:number): number
}

const add: Add = (a, b) => a + b;
```

```
console.log(add(2,5));

// Multiple call signature

interface StringFormatter {
  (input: string): string;
  (input: string, uppercase: boolean): string;
}

const format: StringFormatter = (input: string, uppercase?: boolean) => {
  return uppercase ? input.toUpperCase() : input.toLowerCase();
};

console.log(format("Mahesh")); // "mahesh"
console.log(format("Mahesh", true)); // "MAHESH"
```

## Construct Signatures

```
// Constructor signature are the blue print that tells how objects can be created
with new keyword
//It defines 1. what parameters are passed to constructor and what type of object
is returned after calling new



class Circle {
    constructor(public radius: number) {}
    getArea() {
      return Math.PI * this.radius ** 2;
    }
  }

  interface CircleConstructor {
    new (radius: number): Circle;
  }

  function makeCircle(ctor: CircleConstructor, radius: number) {
    return new ctor(radius);
  }

  const circle = makeCircle(Circle, 10);
  console.log(circle.getArea());
```

## Generic Functions

```
//Generic functions are reusable function that can handle different data type
without loosing type information

function swap<T, U>(a: T, b: U): [U, T] {
    return [b, a];
  }

  const result = swap("Mahesh", 25);
  console.log(result); // [25, "Mahesh"]
```

## Inference

```
// TypeScript can automatically figure out the type based on the value passed in
generic function

function getFirst<T>(arr: T[]): T {
    return arr[0];
  }

  const first = getFirst([10, 20, 30]);  // T inferred as number
  const second = getFirst(["a", "b"]);   // T inferred as string


  function identity<T>(value: T): T {
    return value;
  }

  // Explicit type:
  const a = identity<string>("Mahesh");

  // Inferred type:
  const b = identity("Mahesh");
```

## Constraints

```
// Constraints tell typescript that T must be a type that at least has certain
properties.
// we add constraint using Extend keyword on Type
```

```
// function logLength<T>(item: T): void {
//   console.log(item.length); // Error: Property 'length' does not exist on type
'T'
// }

function logLength<T extends { length: number }>(item: T): void {
    console.log(item.length);
  }

  logLength("Mahesh");
  logLength([1, 2, 3]);
  logLength({ length: 10 });
  // logLength(42);     //Argument of type 'number' is not assignable to
parameter of type '{ length: number; }'
```

## Working with Constrained Values

```
// When working with generic function with constraints --> Function always
returns the Type that has been passed not the constraint

function minimumLength<Type extends { length: number }>(
    obj: Type,
    minimum: number
  ): Type {
    if (obj.length >= minimum) {
      return obj;
    } else {
      return { length: minimum };
  //      Type '{ length: number; }' is not assignable to type 'Type'.
  //   '{ length: number; }' is assignable to the constraint of type 'Type', but
'Type' could be instantiated with a different subtype of constraint '{ length:
number; }'.
    }
  }
```

## Specifying Type Arguments

```
// TypeScript usually infers the generic type automatically, but sometimes you
need to specify it manually using type arguments

function combine<Type>(arr1: Type[], arr2: Type[]): Type[] {
```

```
    return arr1.concat(arr2);
  }

  console.log(combine([1,2,4], [3, 5,6]));  // works fine because both array type
are same
  // console.log(combine([1,2,4], ["Mahesh", "Naneran", "Ronak"]));  // Type
'string' is not assignable to type 'number'



  //fix
  const mixArr = combine<string | number>([1,2,4], ["Mahesh", "Naneran",
"Ronak"]);
  console.log(mixArr);
```

## Guidelines for Writing Good Generic Functions

```
//1.  Push Type Parameters Down
// don’t make the whole function generic unless necessary.

//Bad example
// Here T is too generic — it could be anything (number, boolean, etc.),
// function logLength<T>(arg: T): void {
//   console.log(arg.length); // Error: 'T' might not have a 'length'
// }

//Fix
function logLength<T extends { length: number }>(arg: T): void {
    console.log(arg.length);
  }

  // 2. Use Fewer Type Parameters
  // Don’t use more generic type parameters than necessary.

  //Bad Example
  // function merge<T, U>(a: T[], b: U[]): (T | U)[] {
  //   return a.concat(b);
  // }

  //Fix
  function merge<T>(a: T[], b: T[]): T[] {
    return a.concat(b);
  }
```

```
// 3. Type Parameters Should Appear Twice
// once in the input parameter and once in the output (return type).
//Bad exmaple
function createEmptyArray<T>(): T[] {
  return [];
}

const arr = createEmptyArray(); // T inferred as `never`

// Fix
function wrapInArray<T>(value: T): T[] {
  return [value];
}
```

## Optional Parameters

```
// a function parameter is “optional” when it’s not required to be passed by the
caller.
// When you make the parameter optional. TypeScript set it types to include
undefined
// Optional parameter must appear after all required parameters

function greet(name?: string) {
    console.log(`Hello ${name}`);
  }

  greet("Mahesh"); // Output: Hello Mahesh
  greet();         // Output: Hello undefined

  function log(message: string, userId?: string) {
    const user = userId ?? "Anonymous";
    console.log(`[${user}] ${message}`);
  }

  log("User logged in");    //[Anonymous] User logged in
  log("User logged out", "Mahesh");   //[Mahesh] User logged out
```

## Optional Parameters in Callbacks

```
// When a callback has optional parameters:
// TypeScript allows the caller to provide fewer arguments
```

```
function greetUser(name: string, callback: (message?: string) => void) {
    const message = `Hello, ${name}!`;
    callback(message);
  }

  greetUser("Mahesh", (message) => {
    console.log(message);
  });

  greetUser("Mahesh", () => {
    console.log("Greeting sent successfully!");
  });

  greetUser("Mahesh", (undefined) => {
    console.log("Greeting sent successfully!");
  });
```

## Function Overloads

```
// Function Overloading - Having multiple functions with the same name but
different inputs (parameters).

// You can have multiple overload declarations, but only one implementation.
// The implementation must be broad enough to handle all overload cases.

// Overload signatures
function greet(): string;
function greet(name: string): string;

//Implementation
function greet(name?: string): string {
  if (name) return `Hello, ${name}!`;
  return "Hello, stranger!";
}

console.log(greet());         // Hello, stranger!
console.log(greet("Mahesh")); // Hello, Mahesh!


// Overload signatures
function formatMessage(sender: string, message: string): string;
```

```
function formatMessage(sender: string, message: string, timestamp: Date): string;

// Implementation
function formatMessage(sender: string, message: string, timestamp?: Date): string
{
  if (timestamp) {
    return `[${timestamp.toLocaleTimeString()}] ${sender}: ${message}`;
  }
  return `${sender}: ${message}`;
}

console.log(formatMessage("Mahesh", "Hello there!"));
console.log(formatMessage("Mahesh", "Hello there!", new Date()));
```

## Writing Good Overloads

```
// Prefer Union Types Instead of Many Overloads

// all overloads return the same type
// function len(s: string): number;
// function len(arr: any[]): number;
// function len(x: any): number {
//   return x.length;
// }

function len(x: string | any[]): number {
    return x.length;
  }

  console.log(len("Mahesh"));
  console.log(len([1, 2, 3]))



  // Always Make the Implementation Compatible with All Overloads
  // function show(id: number): void;
  // function show(name: string): void;
  // function show(value: any) {
  //   console.log(value.toUpperCase()); // will crash if value is number
  // }

  function show(id: number): void;
  function show(name: string): void;
```

```
  function show(value: any) {
    if (typeof value === "string") {
      console.log(value.toUpperCase());
    } else {
      console.log("User ID:", value);
    }
  }
```

## Declaring this in a Function

```
// In typescript ,this point to the object that is calling a function

function greet(this: { name: string }) {
    console.log(`Hello, ${this.name}`);
  }

  const user = { name: "Mahesh", greet };
  user.greet(); // "Hello, Mahesh"

  // greet(); // Error in TypeScript — 'this' is missing
```

## Other Types to Know About

### void

```
// void --> when there is nothing to return
function greet(name: string): void {
    console.log(`Hello, ${name}!`);
  }

  greet("Mahesh");
```

### object

```
//object
// the object type represents any non-primitive value.
// object includes array, plain object, functions etc

let person: object;

person = { name: "Mahesh", age: 25 };
```

```
person = [1, 2, 3];
person = function() {};
person = new Date();
// person = "Hello";   // Type 'string' is not assignable to type 'object'.
```

## unknown

```
//unknown
// The unknown type represents any value. This is similar to the any type, but is
safer because it’s not legal to do anything with an unknown value

let data: unknown;

data = 10;
data = "Mahesh";
data = [1, 2, 3];
data = { key: 1 };

let value: unknown = "Hello";
// console.log(value.toUpperCase()); //Property 'toUpperCase' does not exist on
type 'unknown'.
```

## never

```
// never
// This value should never exist.
function processValue(value: string | number) {
    if (typeof value === "string") {
      console.log("String value:", value.toUpperCase());
    } else if (typeof value === "number") {
      console.log("Number value:", value.toFixed(2));
    } else {
      // TypeScript knows this can't happen
      const impossible: never = value;
      console.log(impossible);
    }
  }
```

## Function

```
// Function is a global type that represents any callable object — any function,
regardless of its parameters or return type.
```

```
// You can assign any function to it.
// So it removes type safety!

let greet: Function;

greet = function (name: string) {
  console.log("Hello " + name);
};

greet("Mahesh");
greet(123);
```

## Rest Parameters and Arguments

```
// Rest parameters allow a function to accept any number of arguments as an
array.
// You can have only one rest parameter in a function.
// It must come at the end of the parameter list.

function greetAll(greeting: string, ...names: string[]) {
    for (let name of names) {
      console.log(`${greeting}, ${name}!`);
    }
  }

  greetAll("Hello", "Mahesh", "Ronak", "Shyam");
```

## Rest Arguments

```
// Rest argumnet -- spread operator
// It’s used to expand an array into separate individual arguments.

function add(a: number, b: number, c: number): number {
    return a + b + c;
  }

  const numbers = [1, 2, 3] as const;
  console.log(numbers);

  // Using spread as rest argument
  console.log(add(...numbers));
```

## Parameter Destructuring

```
// Parameter destructuring means unpacking values directly inside the function's
parameter list

interface User {
    name: string;
    age: number;
  }

  function displayUser({ name, age }: User) {
    console.log(`User ${name} is ${age} years old`);
  }

  displayUser({ name: "Mahesh", age: 25 });

  // Array destructuring
  function printCoordinates([x, y]: [number, number]) {
    console.log(`X: ${x}, Y: ${y}`);
  }

  printCoordinates([10, 20]);


  // Default Values with Destructuring
  interface Product{
      name: string,
      price?: number,
      inStock?: boolean
  }

  function showProductDetails({ name:productName, price = 100, inStock = false} :
Product){
      console.log(`Product: ${productName}`);
      console.log(`Price: $${price}`);
      console.log(`In stock: ${inStock ? "Yes" : "No"}`);
  }

  showProductDetails({ name: "Laptop", price: 50000, inStock: false });
```

## Assignability of Functions

```
// Assignability of Functions --> whether one function type can be assigned to
another

type A = (x: number) => void;
type B = () => void;

let a: A;
let b: B;

a = b; // allowed - b doesn’t need 'x'
b = a; // error — a expects a parameter


// A function with return type void can return anything, but the caller must
ignore whatever it returns.

function logMessage(): void {
  console.log("Logging...");
}

function logAndReturn(): void {
  return "Done!"; // Allowed --> It is simply ignored
}

//
type voidFunc = () => void;

const f1: voidFunc = () => {
  return true;
};

const f2: voidFunc = () => true;

const f3: voidFunc = function () {
  return true;
};

const v1 = f1();  // Type Void
const v2 = f2();  // Type Void
const v3 = f3();  // Type Void
```

# Chapter 7: TypeScript - Object Types

## Optional Properties

```
// In object type we can specify the optional property using ? operator

interface Product{
    id: number,
    name: string,
    Description?: string,      // string | undefined
    DiscountPrice?: number     // number | undefined
}

function displayProductDetails(product: Product){
    console.log("Product name:", product.name);
    if(product.DiscountPrice)
    {
        console.log("Discount price:", product.DiscountPrice);
    }
}

const product1: Product = {id: 1, name: "Glass"};
const product2: Product = {id: 2, name: "Watch", DiscountPrice: 300};

displayProductDetails(product1);
displayProductDetails(product2);
```

## readonly Properties

```
// A readonly property means that the property’s value cannot be changed after
it’s initialized.

interface User{
    readonly id: number,
    name: string,
}

let user: User = {
    id: 101,
    name: "Mahesh",
}
```

```
console.log(user.id);
user.name = "Ronak";   // Allowed
// user.id = 30;    // Cannot assign to 'id' because it is a read-only property.

interface Home{
    readonly resident: { name: string, age: number}
}

// we can chnage the property of resisdent
// user.resident.age = 34;    // allowed
// cannot re write the resident property
// resident = {
    // .....               /// Error
// }
```

## Index Signatures

```
// Index signature is used when we do not know the exact property names - but we
knows the type of keys and type of their value
// ex. used when want to create the objcet as dictionaries



interface UserAges{
    [name: string] : number
}

const ages: UserAges = {
    Mahesh: 23,
    Ronak: 22,
    Meet: 35
}

console.log(ages.Mahesh);   // Used as key
console.log(ages["Ronak"]);  // Used as key

// Mixed types
interface UserScores{
    id: number,
    [subject: string]: number
}

const scores: UserScores = {
```

```
    id: 2,
    Maths: 86,
    English: 99
}

// ALl property type must be subtype of index signature value type
//i.e index signature says values are number, then fixed property (id) must also
be a number (not string)

interface UserScores2 {
  id: string;
//   [subject: string]: number; // Property 'id' of type 'string' is not
assignable to 'string' index type 'number'.
}

// Readonly

interface Config {
  readonly [key: string]: string;
}

const settings: Config = {
  theme: "dark",
  version: "1.0"
};

// settings.theme = "light";  // Index signature in type 'Config' only permits
reading.
```

## Extending Types

```
// Extending a type means creating a new type that inherits properties from
another type.

interface Person {
    name: string;
    age: number;
  }

  interface Employee extends Person {
    employeeId: number;
  }
```

```
const emp: Employee = {
  name: "Mahesh",
  age: 25,
  employeeId: 101
};

// Extending Multiple Interfaces
interface Address {
  city: string;
  country: string;
}

interface ContactInfo {
  phone: string;
  email: string;
}

interface User extends Address, ContactInfo {
  name: string;
}

const user: User = {
  name: "Aarav",
  city: "Mumbai",
  country: "India",
  phone: "9876543210",
  email: "aarav@mail.com"
};
```

## Intersection Types

```
// Intersaction types --> another way to extend the types and interfaces using &
operator

interface Person{
    name: string,
    age: number,
}

interface Contact{
    email: string,
```

```
    phone: string,
}

type Employee = Person & Contact;

const employee: Employee = {
    name: "Mahesh",
    age: 23,
    email: "mahesh@temp.com",
    phone: "1234567899"
}

console.log(employee);

// Extending type alias
type Base = { id: number };
type Info = { name: string };
type Extended = Base & Info & { isActive: boolean };

const user2: Extended = {
  id: 1,
  name: "Mahesh",
  isActive: true,
};
```

## Interfaces vs. Intersections

```
// Interface Extension vs. Intersection
// extends = Hierarchical (Parent --> Child)  // Child is kind of parent
// & = Merging   --> Combination of both

// type cannot be extended
// but & combine both type and interface with each other

interface A { id: number; }
interface B { id: string; }

// Both cases cause conflict
// Interface 'C' cannot simultaneously extend types 'A' and 'B'.Named property
'id' of types 'A' and 'B' are not identical.
interface C extends A, B {}
type D = A & B;
```

```
// 2 interfaces with same property combines all the properties if they are
compatible
interface User {
  name: string;
}

interface User {
  age: number;
}

const u: User = { name: "Mahesh", age: 25 };

// if & does not able to find the intersaction in different type with same
property name then it give type to that property never

interface Person1 {
  name: string;
}

interface Person2 {
  name: number;
}

type Staff = Person1 & Person2;

// Staff --> name: never
```

## Generic Object Types

```
// reusable types that work with different data types.

type Box<T> = {
    value: T;
  };

  const stringBox: Box<string> = { value: "Mahesh" };
  const numberBox: Box<number> = { value: 123 };
```

```
interface Container<T> {
  item: T;
}

const stringContainer: Container<string> = { item: "Hello" };
const numberContainer: Container<number> = { item: 123 };
```

## The Array Type

```
let numbers1: number[] = [1, 2, 3, 4];
let names1: string[] = ["Mahesh", "Aarav", "Raj"];



//Array<Type>
let numbers2: Array<number> = [10, 20, 30];
let names2: Array<string> = ["Hello", "World"];

// array with multiple union types
let mixed: (string | number)[] = ["Mahesh", 25, "Aarav", 30];



// Array of object
type User = {
  name: string;
  age: number;
};

let users: User[] = [
  { name: "Mahesh", age: 25 },
  { name: "Aarav", age: 22 }
];
```

## The ReadonlyArray Type

```
// A ReadonlyArray is an array whose elements cannot be changed after creation.

let numbers: ReadonlyArray<number> = [1, 2, 3];
console.log(numbers[0]);
// numbers[1] = 20;   // Index signature in type 'readonly number[]' only permits
reading.
```

```
// numbers.push(4);   // Property 'push' does not exist on type 'readonly
number[]'.

// Alternative syntax
const names: readonly string[] = ["Mahesh", "Aarav"];



// Assignment Mutable --> Readonly
let arr: number[] = [1, 2, 3];
let readonlyArr: ReadonlyArray<number> = arr;



// Readonly --> mutable
// arr = readonlyArr;   // The type 'readonly number[]' is 'readonly' and cannot
be assigned to the mutable type 'number[]'.
```

## Tuple Types

```
// tuple is special array type with fixed number of elements with specific types
in specific positions
// Position with type matters

let person: [string, number, boolean] = ["Mahesh", 25, true];
// person = [true, "Mahesh", 25];   // Type 'boolean' is not assignable to type
'string'.

// Optional element in tuple
let user: [string, number?, boolean?];

user = ["Mahesh"];
user = ["Mahesh", 25];
user = ["Mahesh", 25, true];

// Tuple with rest element (Flexible lenght)
let scores: [string, ...number[]];

scores = ["Math", 90, 85, 88];
scores = ["Science"];

// scores = ["English", "A"]
// '[string, string]' is not assignable to type '[string, ...number[]]'.
```

```
//   Type at position 1 in source is not compatible with type at position 1 in
target.
//     Type 'string' is not assignable to type 'number'.
```

## readonly Tuple Types

```
// readonly tuple is a tuple whose elements cannot be changed

const point: readonly [number, string] = [10, "Mahesh"];

// point[0] = 23;  // Cannot assign to '0' because it is a read-only property
// point.pop()   // Property 'pop' does not exist on type 'readonly [number,
string]'.

function printPoint(point: readonly [number, number]) {
  console.log(`X: ${point[0]}, Y: ${point[1]}`);
//   point[0] = 10; // Cannot assign to '0' because it is a read-only property.
}



// You can make a tuple readonly automatically using as const.

const direction = [10, 20] as const;
// Type of direction → readonly [10, 20]
// Each element is literal (10 and 20, not just number)

// direction[0] = 5;   // Cannot assign to '0' because it is a read-only
property.
```

# Chapter 8: TypeScript - Type Manipulation

## Generics

### Generic function

```
function identity<Type>(value: Type): Type{
    return value;
}

let userName = identity<string>("Mahesh");
let age = identity<number>(23);
```

### Generic type

```
let myIdentity: <Type>(val: Type) => Type = identity;

//using call signature
// interface GenericIdentityFn {
//   <Type>(arg: Type): Type;
// }

interface GenericIdentityFn<Type> {
  (arg: Type): Type;
}
let myIdentity2: GenericIdentityFn<number> = identity;
```

### Generic Classes

```
class CustomClass<Type> {
    zeroValue: Type;
    add: (x: Type, y: Type) => Type;
  }

  let myGenericNumber = new CustomClass<number>();
  myGenericNumber.zeroValue = 0;
  myGenericNumber.add = function (x, y) {
    return x + y;
  };
```

## Generic constraint

```
interface Lengthwise {
    length: number;
  }

  function loggingIdentity<Type extends Lengthwise>(arg: Type): Type {
    console.log(arg.length);
    return arg;
  }

  // loggingIdentity(10); // Argument of type 'number' is not assignable to
parameter of type 'Lengthwise'.
  loggingIdentity("Mahesh");   // 6
```

## Type parametets in Generic constraints

```
// Type parametets in Generic constraints
// One generic type depends on another generic type.
// suppose if we want the object with property name. We like to ensure that we do
not access the property that does not exists in object
// Keyof returns the keys of that object

function getProperty<Type, Key extends keyof Type>(obj: Type, key: Key) {
    return obj[key];
  }
  let x = { a: 1, b: 2, c: 3, d: 4 };

  getProperty(x, "a");
  // getProperty(x, "m");  //Argument of type '"m"' is not assignable to
parameter of type '"a" | "b" | "c" | "d"'.
```

## Using class type in generics

```
// Using class type in generics
// Use classes as generic parameters
// write functions or classes that accept other classes as type arguments.

function createInstance<T>(type: new () => T): T {
    return new type();
  }

  class User {
```

```
    name = "Mahesh";
  }

  class Admin {
    role = "Administrator";
  }

  const user = createInstance(User);
  const admin = createInstance(Admin);

  console.log(user.name);   // Mahesh
  console.log(admin.role);  // Administrator
```

## Generic Default Types

```
class Box<T = string> {
    content: T;

    constructor(content: T) {
      this.content = content;
    }
  }

  const stringBox = new Box("Hello");  // T = string (default)
  const numberBox = new Box<number>(100); // T = number (explicit)

  console.log(stringBox.content); // "Hello"
  console.log(numberBox.content); // 100
```

## Varience Annotation

```

class Vehical{
    brand: string,
}

class Car extends Vehical{
    speed: number
}

interface Box<out T> {
  getValue(): T;
```

```
}

let dogBox: Box<Dog>;
let animalBox: Box<Animal>;

// Covariance --> assignement of subtype to supertype
animalBox = dogBox;

interface Handler<in T> {
  handle(value: T): void;
}

let animalHandler: Handler<Animal>;
let dogHandler: Handler<Dog>;

// Contravarient --> assignemnt of supertype to subtype
dogHandler = animalHandler;
```

## Keyof Type Operator

```
// type operator that takes an object type and produces a union of its keys

type Vehicle = {
    brand: string;
    speed: number;
    isElectric: boolean;
  };

  type VehicleKeys = keyof Vehicle;   // "brand" | "speed" | "isElectric";

  function getValue<T, K extends keyof T>(obj: T, key: K): T[K] {
    return obj[key];
  }

  const car = { brand: "Tesla", speed: 120 };

  const brand = getValue(car, "brand");  // OK
  const speed = getValue(car, "speed");  // OK
  // const color = getValue(car, "color"); // Argument of type '"color"' is not
assignable to parameter of type '"brand" | "speed"'.
```

## Typeof Type Operator

```
// operator that returns a string describing the runtime type of a value.

let message = "Hello";
let age = 10;

console.log(typeof message);
console.log(typeof age);



//ReturnType  --> takes the function type and produces the return type
function f() {
  return { x: 10, y: 3 };
}
type P = ReturnType<typeof f>;
```

## Indexed Access Types

```
// Indexed Access Types --> let you look up the type of a specific property
inside another type

type Person = { age: number; name: string; alive: boolean };
type Age = Person["age"];   // number

type I1 = Person["age" | "name"];   // number | string
type I2 = Person[keyof Person];     // string | number | boolean

const key = "age";
// type Ages = Person[key]; // 'key' refers to a value, but is being used as a
type here. Did you mean 'typeof key'?



// using type alias
type key2 = "age";
type Ages = Person[key2];
```

## Conditional Types

```
// create types that depend on other types
```

```
// Conditional Type  --> T extends U ? X : Y;  --> If T is assignable to U then X
else Y

type Vehicle = { brand: string; speed: number };
type Car = { brand: string; speed: number; doors: number };

type IsVehicle<T> = T extends Vehicle ? "Vehicle Type" : "Not a Vehicle";

type A = IsVehicle<Car>;     // "Vehicle Type"
type B = IsVehicle<string>;  // "Not a Vehicle"

// The infer keyword lets you capture a type from another type when checking
conditions.
type ReturnTypeOf<T> = T extends (...args: any[]) => infer R ? R : never;

function getSpeed() {
  return 120;
}

type Result = ReturnTypeOf<typeof getSpeed>; // number



// Conditional Type with Union
type Check<T> = T extends string ? "String" : "Other";
type TypeResult = Check<string | number>;  // "String" | "Other"
```

## Mapped Types

```
// Mapped types to create new object types by transforming the keys of an
existing type.

type Vehicle = {
    brand: string;
    speed: number;
    electric: boolean;
  };

  type PartialVehicle = {
    [Key in keyof Vehicle]?: Vehicle[Key];
  };
```

```
  /*
  Result:
      type PartialVehicle = {
      brand?: string;
      speed?: number;
      electric?: boolean;
  };
  */

  // Mapping Modifiers  - used to remove readonly or optionality of property
  type CreateMutable<Type> = {
    -readonly [Property in keyof Type]: Type[Property];
  };

  type Concrete<Type> = {
    [Property in keyof Type]-?: Type[Property];
  };

  // Key Remapping via as key --> remapping, you can change the key’s name as you
map it
  type PrefixedVehicle = {
    [K in keyof Vehicle as `vehicle_${K}`]: Vehicle[K];
  };

  /*
  type PrefixedVehicle = {
    vehicle_brand: string;
    vehicle_speed: number;
    vehicle_electric: boolean;
  };

  */
```

## Template Literal Types

```
// Template Literal Types allow you to build string types using template string
syntax

type Greeting = `Hello ${string}`;

let greet1: Greeting = "Hello Mahesh";  // valid
```

```
// let greet2: Greeting = "Hi Mahesh";     // Type '"Hi Mahesh"' is not
assignable to type '`Hello ${string}`'.

type Direction = "top" | "bottom" | "left" | "right";
type Position = "start" | "end";

// TypeScript automatically creates all possible combinations of the literals
type CSSProperty = `${Direction}-${Position}`;

// inference with template literals
// extract or decompose parts of a string literal type that matches a template
literal pattern.

type EventName<T> = T extends `on${infer Name}` ? Name : never;
type A = EventName<"onClick">;   // "Click"
type B = EventName<"onHover">;   // "Hover"
type C = EventName<"Click">;     // never
```

# Chapter 9: TypeScript – Classes

## Class Members

### Fields

```
// Class in typescript is the blueprint to create the object.
// That shares the same structure i.e. Properties and Methods

///Fields/Properties  --> They Stores data
class Student {
  name: string;
  age: number = 18; // default value

  constructor(name: string) {
    this.name = name;
  }
}

const s = new Student("Mahesh");
console.log(s.name, s.age); // Mahesh 18
// s.name = 10;  // Type 'number' is not assignable to type 'string'.


// definite assignment assertion operator  -->  It tells ts that this field is
assigned with value before it's use, even if typescript does not sees this
//  strictPropertyInitialization  --> all the properties either declare while
initialization or in the constructor

class Car {
//   brand: string; // Error: Property 'brand' has no initializer and is not
definitely assigned in the constructor
    brand!: string; // This will be assigned later

  start() {
    console.log(`Starting ${this.brand}`);
  }
}
const c = new Car();
c.brand = "Tesla"; // assigned later
c.start();
```

```
// When to use
// 1. You are absolutely sure you'll assign a value before using it.
// 2. A framework or external library (e.g., Angular) assigns it automatically.
```

## Readonly

```
// readonly
// readonly --> marks class field readonly. value cannot chnaged after
initialization

class Product {
  readonly id: number;
  name: string;
  readonly createdAt: Date;

  constructor(id: number, name: string) {
    this.id = id;
    this.name = name;
    this.createdAt = new Date();
  }

  rename(newName: string) {
    this.name = newName; // allowed
    // this.id = 2  Error
  }
}

const product = new Product(1, "Laptop");
product.rename("Gaming Laptop");
// product.createdAt = new Date();  // annot assign to 'createdAt' because it is
a read-only property.
console.log(product.id, product.name, product.createdAt);
```

## Constructors

```
// Constructor
// constructors--> a special method inside a class that runs automatically when
you create a new object using new
// Initialize properties
// Setup dependencies
// Run setup logic before the object is ready for use.
```

```typescript
class Car {
  brand: string;
  speed: number;

  constructor(brand: string, speed: number) {
    this.brand = brand;
    this.speed = speed;
  }

  drive() {
    console.log(`${this.brand} is driving at ${this.speed} km/h`);
  }
}

const car1 = new Car("Tesla", 120);
car1.drive(); // Tesla is driving at 120 km/h

// Parameter properties shortcut
class Student {
  constructor(public name: string, public age: number) {}
}

const s = new Student("Mahesh", 22);
console.log(s.name, s.age); // Mahesh 22

// automatically declare the properties name and age and assign the values passed
in parameters

//Supercalls

// Super keyword --> used to access and call the methods on object's parent class
// call the parent class constructor --> super(...)
// access parent class method or property  --> super.methodName()
// Before child class uses "this" --> It must call parent class constructor

class Animal {
  constructor(public name: string) {
    console.log(`Animal: ${name}`);
  }
}

class Dog extends Animal {
```

```
  constructor(name: string, public breed: string) {
    super(name);
    console.log(`Dog: ${breed}`);
  }
}

const dog = new Dog("Rocky", "Labrador");
```

## Methods

```
// Methods
class BankAccount {
  balance: number = 0;

  deposit(amount: number) {
    this.balance += amount;
  }

  showBalance() {
    console.log(`Current Balance: ₹${this.balance}`);
  }
}
```

## Getters / Setters

```
// getter and setter
// getter is a special method used to read a property value.
// setter is a special method used to write (update) a property value.

class Student {
  private _age: number = 0;

  get age(): number {
    return this._age;
  }

  set age(value: number) {
    if (value < 0 || value > 120) {
      throw new Error("Invalid age");
    }
    this._age = value;
  }
```

```
}

const s = new Student();
s.age = 25; // works
console.log(s.age); // 25
// s.age = -5; // Error: Invalid age
```

## Index Signatures

```
// An index signature lets you define the type of keys and values that an object
can have when the exact property names are not known in advance.

class Settings {
  [key: string]: string | number | boolean;

  constructor() {
    this["theme"] = "dark";
    this["version"] = 1;
    this["notifications"] = true;
  }
}

const config = new Settings();
console.log(config["theme"]); // dark
console.log(config["version"]); // 1
console.log(config.notifications);
```

## Class Heritage

### Implements Clause

```
// Class Heritage --> How classes can inherits from another class

// Implements clause --> Ensures that the class follows the structure of
interface
interface Flyable {
  fly(): void;
}

interface Swimmable {
  swim(): void;
}

class Duck implements Flyable, Swimmable {
  fly() {
    console.log("Duck is flying!");
  }
  swim() {
    console.log("Duck is swimming!");
  }
}

const duck = new Duck();
duck.fly();
duck.swim();
```

### Extend clause

```
class Animal {
  name: string;

  constructor(name: string) {
    this.name = name;
  }

  move() {
    console.log(`${this.name} is moving`);
  }
}
```

```
class Dog extends Animal {
  // Adding new Method
  bark() {
    console.log(`${this.name} says: Woof!`);
  }

  // Overriding the parent's method
  makeSound() {
    console.log("Woof!");
  }
}

const dog = new Dog("Buddy");
dog.move(); // Buddy is moving  inherited from Animal
dog.bark(); // Buddy says: Woof!  defined in Dog
```

## Overriding

```
// Overriding --> replaces or extends the parent class’s method with the same
name and parameters.

class Animal {
  move() {
    console.log("The animal moves.");
  }
}

class Bird extends Animal {
  move() {
    console.log("The bird flies in the sky.");
  }
}

class Fish extends Animal {
  move() {
    console.log("The fish swims in water.");
  }
}

const bird = new Bird();
const fish = new Fish();
```

```
bird.move();
fish.move();
```

## type-only field declaration

```
// a type-only field declaration is a class property that has only a type — no
initializer, no value assignment — just the type information.
// It exists only for TypeScript’s type checking and disappears in the compiled
JavaScript.
// Assigning Later

class User {
  name!: string; // definite assignment assertion
  age!: number;

  initialize(name: string, age: number) {
    this.name = name;
    this.age = age;
  }
}

const user = new User();
user.initialize("Alice", 30);
console.log(user.name); // "Alice"
```

## Initialization order

```
class Parent {
  fieldParent = console.log("Parent instance field");

  constructor() {
    console.log("Parent constructor");
  }
}

class Child extends Parent {
  fieldChild = console.log("Child instance field");

  constructor() {
    console.log("Child constructor start");
    super(); // Must call super() before accessing this
    console.log("Child constructor end");
```

```
  }
}

const c = new Child();
```

### Extending Built-in types

```
// you can extend (inherit from) built-in types like Array, Error etc. to create
your own specialized versions.

class MyArray<T> extends Array<T> {
  last(): T | undefined {
    return this[this.length - 1];
  }
}

const numbers = new MyArray(1, 2, 3);
console.log(numbers.last()); // 3
```

## Member Visibility

### Access modifiers

```
// Acccess modifiers --> control who can access a class’s properties or methods.
// public --> Exposes the class members outside the class
// protected --> Exposes the class members to subclass.
// private --> does not expose the class members even to the subclasses

class Employee {
  public name: string;
  private salary: number;
  protected department: string;

  constructor(name: string, salary: number, dept: string) {
    this.name = name;
    this.salary = salary;
    this.department = dept;
  }

  public showInfo() {
    console.log(`${this.name} works in ${this.department}`);
  }
```

```
  private calculateTax() {
    return this.salary * 0.2;
  }
}

class Manager extends Employee {
  promote() {
    // this.salary; can't access private
    console.log(`${this.name} from ${this.department} got promoted!`);
  }
}

const emp = new Manager("Mahesh", 60000, "IT");
emp.showInfo();
emp.promote();
// emp.department;     // Error (protected)
// emp.salary;         // Error (private)
```

## Cross-instance private access

```
class A {
  private x = 10;

  public sameAs(other: A) {
    // No error
    return other.x === this.x;
  }
}
```

## Caveats

```
// Caveats
// TypeScript's access modifiers are only for type-checking
// At runtime (in js) they completely erased after compilation

class Example {
  private secret = "hidden";
}

const e = new Example();
```

```
console.log(e["secret"]); // Drawback --> Private field can be access using
bracket notation ... Works at runtime
// console.log(e.secret); // Property 'secret' is private and only accessible
within class 'Example'.

// in compiled js file
// "use strict";
// class Example {
//     constructor() {
//         this.secret = "hidden";
//     }
// }
// const e = new Example();
// console.log(e.secret); // works at runtime

// Strict private notation (#)
// Properties that is declate private using (#) remain private after compilation
also

class User {
  #password: string = "tempPassword";// private field

  constructor(password: string) {
    this.#password = password;
  }

  checkPassword(input: string) {
    return this.#password === input;
  }
}

const user = new User("abc123");

console.log(user.checkPassword("abc123"));
// console.log(user.#password)  // Property '#password' is not accessible outside
class 'User' because it has a private identifier.
console.log(user["#password"])  // Undefined --> Not accessible
```

## Static Members

```
// Static members --> is belongs to class itselt not for individual object or
instances
```

```
class MyClass {
  static propertyName = "I belong to the class";

  static methodName() {
    return "Called from the class";
  }
}

console.log(MyClass.propertyName); // "I belong to the class"
console.log(MyClass.methodName()); // "Called from the class"

const obj = new MyClass();
// console.log(obj.propertyName); //Property 'propertyName' does not exist on
type 'MyClass'. Did you mean to access the static member 'MyClass.propertyName'
instead?


class Counter {
  count = 0;
  static total = 0;

  constructor() {
    Counter.total++;
  }

  increment() {
    this.count++;
  }

  static getTotal() {
    return Counter.total;
  }
}

const a = new Counter();
const b = new Counter();

a.increment();
b.increment();
b.increment();

console.log(a.count);         // 1
```

```
console.log(b.count);         // 2
console.log(Counter.getTotal()); // 2 (static shared count)
```

## static Blocks in Classes

```
// A static {} block is a special section inside a class that lets you run
initialization code for static members.
// It runs once, when the class is first loaded.
// It can access and modify static fields.
class MyClass {
  static value: number;

  // Static block
  static {
    console.log("Static block is running...");
    MyClass.value = Math.random();
  }
}

console.log(MyClass.value);
```

## Generic Classes

```
// A Generic Class is a class that can work with different data types

class Box<T> {
  content: T;

  constructor(value: T) {
    this.content = value;
  }

  getContent(): T {
    return this.content;
  }
}

const stringBox = new Box("Hello");
const numberBox = new Box(123);
```

```
//  Static members --> are not allowed to references the Type parameters as their
instances are not going to created
class Animal<Type> {
//   static defaultValue: Type;  // Static members cannot reference class type
parameters.
}
```

## this

### this at Runtime in Classes

```
// In JavaScript (and TypeScript), this is a runtime binding — its value depends
on how a function is called, not where it’s defined.

class MyClass {
  name = "MyClass";

  getName() {
    return this.name;
  }
}

const c = new MyClass();
console.log(c.getName()); // "MyClass" --> Here this refers to the instance c

const obj = {
  name: "obj",
  getName: c.getName,
};

console.log(obj.getName());  // obj --> here this refers to the obj as function
is called through obj
```

### using arrow functions

```
// Fix -- Using Arrow Function
// Arrow functions don’t create their own this, they “capture” the this from
where they are defined.

class MyClass2 {
  name = "MyClass";
```

```
  getName = () => {
    return this.name;
  };
}

const c2 = new MyClass2();
const obj1 = { name: "obj", getName: c2.getName };

console.log(obj1.getName()); // "MyClass"


// arrow function uses more memory
class WithPrototypeMethod {
  getName() {
    return "Class";
  }
}

class WithArrowMethod {
  getName = () => {
    return "Class";
  };
}

const a = new WithPrototypeMethod();
const b = new WithPrototypeMethod();

console.log(a.getName === b.getName); //true (shared function)

const x = new WithArrowMethod();
const y = new WithArrowMethod();

console.log(x.getName === y.getName); // false (each has its own copy)


// Cannot user super.methodName for arrow function
class Base {
  greet = () => {
    console.log("Hello from Base");
  };
}
```

```
class Derived extends Base {
  greet = () => {
    //   super.greet(); // Class field 'greet' defined by the parent class is not
accessible in the child class via super.
    console.log("Hello from Derived");
  };
}
```

## this parameter

```
// this parameter --> tell TypeScript what kind of value this is expected to be
inside a function or method.
// this parameters is erased at runtime
class MyClass {
  name = "TypeScript";

  greet(this: MyClass) {
    console.log(`Hello from ${this.name}`);
  }
}

const c = new MyClass();
c.greet(); //Works

const greetFn = c.greet;
// greetFn(); // The 'this' context of type 'void' is not assignable to method's
'this' of type 'MyClas2s'.
```

## this type

```
// this type
// A this type is a special TypeScript type that refers to the current class or
object type where it’s used

class Box {
  content = "";

  set(value: string): this {
    this.content = value;
    return this;
  }
```

```
  sameAs(other: this) {
    return other.content === this.content;
  }
}

class ClearableBox extends Box {
  clear() {
    this.content = "";
    return this;
  }
}

const box = new ClearableBox();
box.set("hello").clear();  // works because set return type of current class i.e
ClearableBox

const base = new Box();
const derived = new DerivedBox();
// derived.sameAs(base);  // Argument of type 'Box' is not assignable to
parameter of type 'DerivedBox'. Property 'otherContent' is missing in type 'Box'
but required in type 'DerivedBox'.
```

## this-type based off-guard

```
// this-based type off-guards
// this-based type guard is a method inside a class that tells TypeScript --> If
this method returns true, then this is of a more specific type.

class Shape {
  kind: string = "shape";

  isCircle(): this is Circle {
    return this.kind === "circle";
  }
}

class Circle extends Shape {
  kind = "circle";
  radius: number;

  constructor(radius: number) {
    super();
```

```
    this.radius = radius;
  }

  area() {
    return Math.PI * this.radius ** 2;
  }
}

const shape: Shape = new Circle(5);

if (shape.isCircle()) {
  // TypeScript now knows `shape` is a Circle
  console.log("Area:", shape.area());
} else {
  console.log("Not a circle");
}
```

## Parameter Properties

```
// Parameter Properties

// declare and initialize class fields directly inside the constructor parameters
— instead of writing them separately.

// constructor(public name: string)
// Adds a class property named name.
// Adds the assignment this.name = name inside the constructor body.

class Product {
  constructor(
    public name: string,
    private price: number,
    discount: number // normal parameter, not a class property
  ) {
    console.log(`${name} has a discount of ${discount}%`);
  }
}

const p = new Product("Phone", 800, 10);

console.log(p.name); //
```

```
// console.log(p.price); // Property 'price' is private and only accessible
within class 'Product'.
// console.log(p.discount); //  Property 'discount' does not exist on type
'Product'.
```

## Class Expressions

```
// Class Expression is a way to define a class as part of an expression
// you can create a class inline, often without a name, and assign it to a
variable.
// class { ... } is a class expression
// defining short-lived or dynamic classes

const Person = class {
  name: string;

  constructor(name: string) {
    this.name = name;
  }

  sayHi() {
    console.log(`Hi, I'm ${this.name}`);
  }
};

const p = new Person("Mahesh");
p.sayHi();


// Inline Class Expression
function createGreeter() {
  return class {
    greet() {
      console.log("Hello!");
    }
  };
}

const Greeter = createGreeter();
const g = new Greeter();
g.greet();
```

```
// we can also extend the class in inline expression
```

## abstract Classes and Members

```
// An abstract class is a base class that cannot be instantiated directly.
// It’s meant to be inherited by other classes.
// The role of abstract classes is to serve as a base class for subclasses which
do implement all the abstract members.

abstract class Animal {
  constructor(public name: string) {}
abstract makeSound(): void; // Abstract method (no implementation)

move(): void {
  console.log("Moving along!");
}
}

// const animal = new Animal();  // Cannot create an instance of an abstract
class.

// Non-abstract class 'Dog' does not implement inherited abstract member
makeSound from class 'Animal'.

class Dog extends Animal {
makeSound(): void {
  console.log("Woof! Woof!");
}
}

const dog = new Dog("Puppy");
dog.makeSound();
dog.move();
```

## Relationships Between Classes

```
//  classes in TypeScript are compared structurally, the same as other types.
class Point {
  x: number = 0;
  y: number = 0;
}
```

```
class Coordinate {
  x: number = 0;
  y: number = 0;
}

let p: Point = new Coordinate(); // Allowed
let c: Coordinate = new Point(); // Allowed

// Only instance members matter for structural comparison.

class A {
  value = 1;
  constructor(x: number) {}
}

class B {
  value = 1;
  constructor() {}
}

let a: A = new B(); // OK — instance shape matches

// If a class has private or protected members, then only that class and its
subclasses are considered compatible.
```

# Chapter 10: TypeScript – Modules

## Modules in TypeScript

```
// modules is the piece of code that is shared via import and export
//modules in TypeScript

//Default export
// @filename: hello.ts
export default function helloWorld() {
  console.log("Hello, world!");
}

import helloWorld from "./hello.js";
helloWorld();

// we can have more than one export
// @filename: maths.ts
export var pi = 3.14;
export let squareTwo = 1.41;
export const phi = 1.61;

export class RandomNumberGenerator {}

export function absolute(num: number) {
  if (num < 0) return num * -1;
  return num;
}

// import { pi, phi, absolute } from "./maths.js";
import RandomNumberGenerator, { pi as π } from "./maths.js"; // we can import
both default and other functions like this

console.log(pi);
const absPhi = absolute(phi);

// Rename the variable that has been imported
import { pi as π } from "./maths.js";
console.log(π);
```

```
// we take all of the exported objects and put them into a single namespace using
* as name
// @filename: app.ts
import * as math from "./maths.js";

console.log(math.pi);
const positivePhi = math.absolute(math.phi);

// Import type
// @filename: animal.ts
export type Cat = { breed: string; yearOfBirth: number };

export interface Dog {
  breeds: string[];
  yearOfBirth: number;
}

// @filename: app.ts
import { Cat, Dog } from "./animal.js";
type Animals = Cat | Dog;

// shorthand syntax for import type
// @filename: app.ts
import { createCatName, type Cat, type Dog } from "./animal.js";

export type Animals = Cat | Dog;
const name = createCatName();
```

## CommonJS Syntax

```
// CommonJS is a module system created for JavaScript
// It is mainly used to in node.js
// Export values from one module
// Import them into another file
// Each file is treated as a separate module
// Anything defined inside a file is scoped to that file
// module.exports → to export
// require() → to import

// math.js
function add(a, b) { return a + b; }
function subtract(a, b) { return a - b; }
```

```
const pi = 3.14;

module.exports = {
  add,
  subtract,
  pi
};

// app.js
const math = require('./math');

console.log(math.add(2, 3));      // 5
console.log(math.subtract(5, 2)); // 3
console.log(math.pi);
```

## TypeScript’s Module Resolution Options

```
// Module Resolution -->
// Two types of module resolution --> 1. Classic, 2. Node
// Classic --> Used only when no module system like Node’s is expected



// tsconfig.ts
// {
//   "compilerOptions": {
//     "moduleResolution": "node"   // or "classic"
//   }
// }

// Module Resolution Flow
// import X from "somePath"
//          ↓
// Check if "somePath" is a relative or non-relative path
//          ↓
// If relative → check nearby files (./, ../)
// If non-relative → search node_modules up the directory tree
//          ↓
// If found → load it
// If not → error: Cannot find module
```

## TypeScript namespaces

```
// A namespace in TypeScript is a way to group related code together under a
single name.
// Members marked with export are public
// Members without export are private to the namespace

namespace Auth {
  const secretKey = '12345'; // private

  export function login(user: string) {
    console.log(`${user} logged in.`);
  }
}

Auth.login('Mahesh'); // works
// Auth.secretKey;    // Error: Property 'secretKey' does not exist on type
'typeof Auth'.
```

## Import Module

```
// Import Module
// math.js
export const PI = 3.14;
export function add(a, b) { return a + b; }

// app.js
import { add, PI } from './math.js';
```

## Export module - Named Export and Default Export

```
// Export
// Default Export

// logger.js
export default function log(message) {
  console.log('LOG:', message);
}

// app.js
import log from './logger.js'; // no curly braces needed
log('Hello!');
```

```
// Named Export
// math.js
export const PI = 3.14;
export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }

// app.js
import { add, PI } from './math.js';



// Mixed Export
// utils.js
export default function greet() {
  console.log("Hello!");
}
export function bye() {
  console.log("Goodbye!");
}

// app.js
import greet, { bye } from './utils.js';
greet();
bye();
```

# Chapter 11: TypeScript – Interfaces

## Interfaces syntax

```
// Interfaces
// An interface in TypeScript defines the structure of an object — what
properties it must have, and what types those properties should be.

interface Person {
  name: string;
  age: number;
}

const user: Person = {
  name: "Mahesh",
  age: 25
};
```

## Optional Property

```
// Optional Properties
interface User {
  name: string;
  email?: string; // optional
}

const u1: User = { name: "Mahesh" };          // valid
const u2: User = { name: "Ravi", email: "ravi@mail.com" }; // valid
```

## Readonly

```
// readonly properties
interface Car {
  readonly model: string;
  speed: number;
}

const myCar: Car = { model: "BMW", speed: 200 };
myCar.speed = 250;   // allowed
// myCar.model = "Audi"; // Error: cannot modify readonly property
```

## Function Type

```
// Function Types in Interfaces
interface Add {
  (a: number, b: number): number;
}

const add: Add = (x, y) => x + y;
console.log(add(5, 10)); // 15
```

## Index Signature

```
// Index Signatures
interface StringDictionary {
  [key: string]: string;
}

const colors: StringDictionary = {
  red: "#ff0000",
  blue: "#0000ff"
};

console.log(colors["red"]); // "#ff0000"
```

## Extending Interfaces

```
interface Animal {
  name: string;
}

interface Dog extends Animal {
  breed: string;
}

const pet: Dog = {
  name: "Rocky",
  breed: "German Shepherd"
};
```

## Interface Merging

```
interface Student {
```

```
  name: string;
}

interface Student {
  age: number;
}

const s: Student = {
  name: "Mahesh",
  age: 22
};
```

## Interfaces for Classes

```
interface Drivable {
  start(): void;
  stop(): void;
}

class Car implements Drivable {
  start() {
    console.log("Car started");
  }
  stop() {
    console.log("Car stopped");
  }
}

const c = new Car();
c.start();
c.stop();
```

## Extending Classes with Interfaces

```
class Animal {
  name: string = "Unnamed";
}

interface Dog extends Animal {
  breed: string;
}
```

```typescript
const dog: Dog = {
  name: "Bruno",
  breed: "Labrador"
};
```

## Interfaces with Generics

```typescript
interface Box<T> {
  value: T;
}

const stringBox: Box<string> = { value: "Hello" };
const numberBox: Box<number> = { value: 100 };
```